

Unit A3: Databases (Core) - Expanded Guide

Sub-topics: A3.1 to A3.3 (SL Content)

1. Syllabus Scope & Learning Outcomes

The "Do" column defines the practical skills required for the exam (Paper 1). derived directly from the textbook text.

Syllabus Point	Command Term	Student Expectation (The "Do")
A3.1.1	Explain	Explain features (Keys, Tables, Relationships), Benefits (Integrity, Concurrency, Security), and Limitations (Scale, Complexity) of relational databases.
A3.2.1	Describe	Describe the three schema levels: Conceptual (High-level entities), Logical (Keys/Attributes), and Physical (DBMS implementation).
A3.2.2	Construct	Construct ERDs (Entity Relationship Diagrams) using correct notation for Entities (Rectangles), Attributes (Ovals/List), and Relationships (Diamonds/Lines).
A3.2.3	Outline	Outline specific data types: Integer, Float, Varchar, Date, Boolean, and Currency. Explain the impact of choosing the

		wrong type.
A3.2.4	Construct	Construct normalized tables ensuring Referential Integrity (Foreign Keys must exist in the parent table).
A3.2.5	Explain	Explain Normal Forms using specific terms: Atomicity (1NF), Partial Dependency (2NF), Transitive Dependency (3NF).
A3.2.6	Construct	Normalise a flat-file dataset (e.g., a spreadsheet view) into a 3NF relational schema.
A3.2.7	Evaluate	Evaluate Denormalization : deliberately adding redundancy to speed up read operations at the cost of write performance/consistency.
A3.3.1	Outline	Distinguish between DDL (Defining structures: CREATE, DROP) and DML (Manipulating data: SELECT, INSERT).
A3.3.2	Construct	Write complex SELECT queries involving WHERE clauses, AND/OR logic, and INNER JOIN across multiple tables.
A3.3.3	Explain	Explain the syntax and risk of UPDATE and DELETE commands (e.g., forgetting a WHERE clause deletes everything).

2. Key Terminology & Definitions (Textbook Specific)

- **Relational Database:** Data organized into tables (relations) linked by common fields (keys).
- **Entity:** A distinct object (person, place, thing) to be tracked.
- **Tuple:** A single row or record in a table.
- **Attribute:** A column or field defining a property of the entity.
- **Candidate Key:** Any column that *could* uniquely identify a row.
- **Primary Key (PK):** The chosen Candidate Key that uniquely identifies a record. Cannot be NULL.
- **Foreign Key (FK):** A column in one table that refers to the PK of another table. Enforces relationships.
- **Composite Key:** A PK made of two or more columns (common in link tables).
- **Schema:** The structural description of the database.
- **Normalization:** The systematic process of reducing data redundancy and anomalies (Update, Insertion, Deletion anomalies).
- **Cardinality:** The numerical nature of a relationship: 1:1 (One-to-One), 1:M (One-to-Many), M:M (Many-to-Many).
- **Modality:** Whether a relationship is mandatory (must exist) or optional (can be null).
- **SQL:** Structured Query Language.
- **DDL:** Data Definition Language (Structure).
- **DML:** Data Manipulation Language (Content).

3. Core Content & Exam Actions

A3.1: Fundamentals

Theory:

- **Why Relational?**
 - **Data Integrity:** Rules ensure data is valid (e.g., you can't sell a product that doesn't exist).
 - **Reduced Redundancy:** Data is stored in one place only (Normalization).
 - **Security:** Access rights can be set per table or view.
- **Limitations:**
 - **Scalability:** Hard to scale across multiple servers (Horizontal scaling).
 - **Complexity:** Designing a correct schema is difficult.
 - **Rigidity:** Changing the schema later can break applications.

Student Exam Action:

- **Scenario:** A school stores student data in spreadsheets. Explain **two** problems they might face (Data Inconsistency, Update Anomalies) and how a Database solves them.

A3.2: Design & Normalization

Theory:

- **ERD Rules:**
 - **1:M:** Place the FK in the "Many" side table.
 - **M:M:** Create a new "Link Table" (Junction Table) with two 1:M relationships.
- **Normal Forms (The Rules):**
 - **1NF:** Eliminate repeating groups. Ensure atomic values (no comma-separated lists).
 - **2NF:** Must be in 1NF. Eliminate **Partial Dependencies** (Attributes must depend on the *whole* Primary Key, not just part of a composite key).
 - **3NF:** Must be in 2NF. Eliminate **Transitive Dependencies** (Attributes must depend on the Key, the Whole Key, and Nothing but the Key).

Student Exam Action:

- **Normalization Task:** Given a table Order(OrderID, Customer, Product, CustomerAddress), normalize it to 3NF. (Move CustomerAddress to a separate Customer table to fix the Transitive Dependency).

A3.3: SQL Programming

Theory:

- **Query Structure:** SELECT [Columns] FROM [Table] WHERE [Condition] ORDER BY [Sort].
- **Joins:** SELECT * FROM TableA JOIN TableB ON TableA.id = TableB.a_id.
- **Wildcards:** % (Multiple characters), _ (Single character) used with LIKE.
- **Logical Operators:** AND, OR, NOT, BETWEEN.

Student Exam Action:

- **Writing Code:** Write a query to select the Name and Price of all products in the "Electronics" category that cost more than \$500.
SELECT Name, Price
FROM Products
WHERE Category = 'Electronics' AND Price > 500;

4. Common Pitfalls

- **Pitfall (M:M):** Trying to put a foreign key in *both* tables for a Many-to-Many relationship. This creates a circular dependency. Always use a third table.
- **Pitfall (Data Types):** Storing dates as text strings (e.g., "Jan 1st"). Prevents sorting and date math. Use the DATE type.
- **Pitfall (Delete):** Running DELETE FROM Students removes *all* records. Always specify WHERE ID = X.